Rubrika - Proyecto Grupal Backend · Bootcamp Full Stack

MEMORIA TÉCNICA

Rubrika

Autoría: Montse García · Saray Guillen · Olatz González · Jonathan Vélez

Fecha: Abril 2026

Repositorio: github.com/r3dc0m/rubrika

Índice

Índice....1
1. Introducción y Contexto....2
1.1 Antecedentes y Definición del Problema....2
1.2 Justificación....2
2. Análisis de Requisitos....3
2.1 Requisitos Funcionales (RF)....3
2.2 Requisitos No Funcionales (RNF)....3
3. Sistema de Autenticación....4
3.1 Registro y acceso a la plataforma....4
3.2 Comprobación de los datos del formulario....4
3.3 Mensajes de aviso y orientación al usuario....4
3.4 Sesión activa mientras se navega....5
4. Arquitectura y Modelo de Datos....6
4.1 Modelo de Datos y Relaciones....6
4.2 Diseño de la API (Endpoints Principales)....6
4.3 Stack Tecnológico y Justificación....7
5. Retos Técnicos y Soluciones....8
5.1 Incompatibilidad con Express v5....8
5.2 Variable de entorno incorrecta en db.js....8
5.3 Datos persistentes de Docker bloqueando la reinicialización....8
5.4 Conflicto de puerto 3000....8
6. Conclusiones y Futuros Pasos....11
Mejoras para la v2.0....11

1. Introducción y Contexto

1.1 Antecedentes y Definición del Problema

En el sector educativo actual, y específicamente en formaciones intensivas tipo bootcamp, la evaluación continua de proyectos es un proceso crítico. Sin embargo, la gestión de esta evaluación se enfrenta a graves problemas:

- Fragmentación: El proceso depende de herramientas desconectadas (hojas de cálculo para notas, email para feedback, formularios para recoger datos).
- Falta de trazabilidad: No existe un registro centralizado que vincule criterios de evaluación, proyectos y notas finales de forma coherente.
- Carga administrativa: El profesorado invierte tiempo en gestión manual en lugar de en la corrección.

1.2 Justificación

Rubrika nace para cubrir esta necesidad. Es una plataforma web diseñada específicamente para la evaluación por criterios ponderados en entornos educativos. A diferencia de otras plataformas generalistas, Rubrika prioriza la claridad y la trazabilidad: permite crear tareas con criterios de evaluación personalizados, asociar proyectos del alumnado y realizar evaluaciones detalladas criterio a criterio, garantizando la integridad de los datos y un flujo de trabajo ágil tanto para el profesorado como para el alumnado.

2. Análisis de Requisitos

2.1 Requisitos Funcionales (RF)

El sistema debe permitir las siguientes interacciones clave:

ID	Requisito	Descripción
RF-01	Gestión de Usuarios (RBAC)	El sistema diferenciará entre roles de Profesor y Alumno con permisos diferenciados.
RF-02	Autenticación	Registro y login funcional con contraseña hasheada mediante bcrypt y sesiones seguras.
RF-03	Gestión de Tareas	El profesorado puede crear tareas con nombre, fecha de entrega, ponderación y activación de evaluación.
RF-04	Criterios de Evaluación	El profesorado puede definir criterios personalizados con peso específico para cada tarea.
RF-05	Gestión de Proyectos	Asociación de proyectos del alumnado a tareas, incluyendo enlace al repositorio.
RF-06	Evaluaciones Detalladas	Calificación por criterio con comentarios generales para cada evaluación.
RF-07	Vistas navegables	Navegación funcional mediante motor de plantillas Pug con layout compartido.
RF-08	API RESTful	Arquitectura de API independiente con endpoints para todas las entidades del sistema.

2.2 Requisitos No Funcionales (RNF)

Restricciones técnicas y de calidad que rigen el desarrollo:

ID	Requisito	Descripción
RNF-01	Seguridad	Las contraseñas se almacenan hasheadas con bcrypt. Las sesiones se gestionan con express-session y clave secreta en variables de entorno.
RNF-02	Integridad de Datos	El modelo relacional implementa Foreign Keys para garantizar la integridad referencial entre todas las entidades.
RNF-03	Responsive y Accesible	La interfaz cumple el estándar de accesibilidad AA con contrastes mínimos de 4.5:1 y es totalmente responsive.
RNF-04	Configuración por entorno	Toda la configuración sensible se gestiona mediante variables de entorno y nunca se sube al repositorio.
RNF-05	Contenedores Docker	La base de datos y pgAdmin se ejecutan en contenedores Docker para garantizar la reproducibilidad del entorno.

3. Sistema de Autenticación

El sistema de autenticación permite a los usuarios acceder a la plataforma de forma segura. Cubre todo el flujo desde el registro hasta el cierre de sesión, incluyendo la validación de los datos introducidos y los mensajes de aviso cuando algo no es correcto.

Funcionalidad	Estado
Registro y acceso a la plataforma mediante usuario y contraseña.	Completado
Comprobación de los datos introducidos en los formularios antes de enviarlos.	Completado
Mensajes de aviso claros cuando los datos son incorrectos o falta información.	Completado
La sesión del usuario se mantiene activa mientras navega por la plataforma.	Completado

3.1 Registro y acceso a la plataforma

Para poder usar Rubrika, cada persona necesita tener una cuenta. El proceso es sencillo: se accede al formulario de registro, se introducen los datos personales y se crea la cuenta. A partir de ese momento, el usuario puede iniciar sesión con su correo y contraseña.

Por seguridad, la contraseña nunca se guarda tal como la escribe el usuario. Antes de almacenarla en la base de datos, se transforma mediante un proceso de cifrado que hace imposible recuperar la contraseña original aunque alguien acceda a los datos. Esto protege a los usuarios incluso en el peor escenario posible.

3.2 Comprobación de los datos del formulario

Antes de procesar cualquier formulario, el sistema verifica que la información introducida es correcta. Esta comprobación se realiza en dos momentos:

En el navegador (antes de enviar)

- Se comprueba que ningún campo obligatorio esté vacío.
- Se verifica que el correo electrónico tiene el formato correcto.
- Se comprueba que la contraseña tiene la longitud mínima requerida.

En el servidor (después de enviar)

- Se verifica que el correo no esté ya registrado por otro usuario.
- Se comprueba que la contraseña y su confirmación coinciden.
- Se valida que todos los datos tienen el formato esperado antes de guardarlos.

3.3 Mensajes de aviso y orientación al usuario

Cuando algo no es correcto, la plataforma informa al usuario de forma clara y precisa, sin tecnicismos ni mensajes confusos. Por ejemplo, si el correo o la contraseña no son correctos al intentar iniciar sesión, el sistema muestra un mensaje genérico indicando que las credenciales son incorrectas, sin especificar cuál de los dos datos falla. Esto es una práctica de seguridad habitual que evita que alguien pueda descubrir si un correo concreto está registrado en la plataforma.

Los errores de cada campo del formulario aparecen justo debajo del campo correspondiente, de modo que el usuario sabe exactamente qué tiene que corregir sin tener que buscar el problema.

3.4 Sesión activa mientras se navega

Una vez que el usuario inicia sesión correctamente, la plataforma recuerda quién es mientras navega por las diferentes secciones. No es necesario volver a introducir el usuario y la contraseña en cada página.

Esta sesión tiene una duración máxima de 24 horas por seguridad. Pasado ese tiempo, o si el usuario cierra sesión manualmente, deberá volver a identificarse. Además, las rutas de la plataforma que requieren estar identificado comprueban automáticamente si existe una sesión activa y, en caso contrario, redirigen al formulario de acceso.

4. Arquitectura y Modelo de Datos

4.1 Modelo de Datos y Relaciones

La aplicación sigue el patrón MVC (Modelo-Vista-Controlador): los modelos gestionan los datos con Sequelize ORM y PostgreSQL, los controladores contienen la lógica de negocio, y las vistas se renderizan con el motor de plantillas Pug. El modelo de datos cuenta con 8 tablas relacionadas:

Tabla	Descripción	Relaciones principales
users	Usuarios del sistema con rol asignado.	1:M con evaluations, M:M con projects
tasks	Tareas creadas por el profesorado.	1:M con projects, M:M con criteria
projects	Proyectos del alumnado asociados a tareas.	M:1 con tasks, 1:M con evaluations
criteria	Criterios de evaluación reutilizables.	M:M con tasks y evaluations
evaluations	Evaluaciones de proyectos.	M:1 con users y projects, M:M con criteria
task_criteria	Tabla intermedia tasks-criteria con peso.	FK a tasks y criteria
user_projects	Tabla intermedia users-projects.	FK a users y projects
evaluations_criteria	Notas por criterio en cada evaluación.	FK a evaluations y criteria

4.2 Diseño de la API (Endpoints Principales)

Se ha diseñado una API RESTful independiente siguiendo las convenciones de nomenclatura de recursos:

Método	Endpoint	Descripción	Protegido
POST	/api/auth/register	Registro de nuevo usuario.	No
POST	/api/auth/login	Autenticación y creación de sesión.	No
GET	/api/users	Lista todos los usuarios.	Sí
GET	/api/tasks	Lista todas las tareas.	Sí
POST	/api/tasks	Crea una nueva tarea (solo profesores).	Sí
PUT	/api/tasks/:id	Actualiza una tarea existente.	Sí
DELETE	/api/tasks/:id	Elimina una tarea.	Sí
GET	/api/projects	Lista todos los proyectos.	Sí
POST	/api/projects	Crea un nuevo proyecto.	Sí
GET	/api/evaluations	Lista todas las evaluaciones.	Sí
POST	/api/evaluations	Crea una nueva evaluación.	Sí
GET	/api/criteria	Lista todos los criterios.	Sí
POST	/api/criteria	Crea un nuevo criterio.	Sí

4.3 Stack Tecnológico y Justificación

Tecnología	Justificación
Node.js + Express 4	Arquitectura no bloqueante ideal para múltiples peticiones simultáneas. Se eligió Express v4 sobre v5 por su mayor madurez y estabilidad.
PostgreSQL 16	Base de datos relacional necesaria para mantener la integridad referencial entre todas las entidades mediante Foreign Keys.
Sequelize ORM	Facilita la definición de modelos y relaciones en JavaScript, abstrayendo las queries SQL y permitiendo sincronización controlada.
Pug	Motor de plantillas con herencia de layouts, permitiendo una estructura HTML consistente con mínima repetición de código.
bcrypt + express-session	bcrypt para el hashing seguro de contraseñas. express-session para la gestión del estado de autenticación entre peticiones.
Docker + Docker Compose	Garantiza la reproducibilidad del entorno de base de datos, evitando problemas de configuración entre distintas máquinas.

5. Retos Técnicos y Soluciones

5.1 Incompatibilidad con Express v5

Situación: Al instalar las dependencias, npm instaló Express v5 (5.2.1) automáticamente. Esta versión provocaba que el servidor arrancase pero no respondiera a ninguna petición HTTP.

Investigación: Tras descartar problemas de red, Docker y variables de entorno, se identificó que el proceso se cerraba tras arrancar en lugar de quedarse escuchando.

Solución implementada:

```
npm install express@4
```

5.2 Variable de entorno incorrecta en db.js

Situación: La aplicación no podía conectarse a la base de datos pese a que las credenciales en el .env eran correctas. Error: password authentication failed for user 'user_rubrika'.

Investigación: Se verificó que el contenedor Docker tenía las variables correctas. El problema residía en el código de configuración de Sequelize.

Solución implementada: Se detectó que en src/config/db.js se leía process.env.DB_PASS en lugar de process.env.DB_PASSWORD.

5.3 Datos persistentes de Docker bloqueando la reinicialización

Situación: Al reiniciar los contenedores con docker compose down -v, la base de datos no se reinicializaba y los scripts SQL no se ejecutaban.

Causa: El proyecto usa un bind mount (./db/data) en lugar de un volumen Docker nombrado. El flag -v solo elimina volúmenes nombrados, no bind mounts.

Solución implementada:

```
docker compose downsudo rm -rf db/data/*docker compose up -d
```

5.4 Conflicto de puerto 3000

Situación: Al arrancar el servidor se producía el error EADDRINUSE: address already in use 0.0.0.0:3000.

Causa: Un proceso anterior del servidor seguía en ejecución en segundo plano ocupando el puerto.

Solución implementada:

```
lsof -i :3000kill -9 <PID>
```

5.5 Implementación del CRUD de Tareas

La gestión completa de tareas constituye una de las funcionalidades centrales de Rubrika, permitiendo al profesorado crear, modificar y eliminar tareas de forma estructurada y protegida mediante un sistema de autorización por roles.

Arquitectura y separación de responsabilidades. El CRUD de tareas sigue el patrón MVC completo con separación clara de responsabilidades:

- Modelo (modelTasks.js): Define el esquema de la tabla tasks y sus relaciones mediante Sequelize ORM.

- Controlador (controllerTasksViews.js): Contiene la lógica de negocio con las funciones getTasks, createTask, updateTask y deleteTask.

- Vista (tasks/list.pug, tasks/form.pug): Renderiza la interfaz de usuario utilizando el motor de plantillas Pug con herencia de layouts.

- Rutas (viewRouterTasks.js): Define los endpoints HTTP y aplica los middlewares de protección por rol.

Sistema de Protección por Roles

Se implementó el middleware requireRoleView en middlewareAuth.js para proteger las rutas de creación, edición y eliminación de tareas. Este middleware intercepta las peticiones HTTP y verifica que el usuario autenticado tenga el rol profesor antes de permitir el acceso:

```
router.get('/new', requireRoleView('profesor'), getCreateTaskForm);
router.post('/new', requireRoleView('profesor'), createTask);
router.get('/:id/edit', requireRoleView('profesor'), getEditTaskForm);
router.post('/:id/edit', requireRoleView('profesor'), updateTask);
router.post('/:id/delete', requireRoleView('profesor'), deleteTask);
```

Los alumnos pueden visualizar la lista completa de tareas mediante la ruta /tasks, pero la interfaz no muestra los botones de acción (crear, editar, borrar) para usuarios con rol alumno. Esta doble capa de protección —tanto en el cliente como en el servidor— garantiza la seguridad del sistema.

Validaciones Implementadas

El sistema incorpora tres capas de validación:

Validación de integridad referencial: Antes de eliminar una tarea, el sistema consulta la tabla evaluations para verificar que no existan evaluaciones asociadas. Si se detectan evaluaciones vinculadas, la operación de borrado se rechaza y se devuelve un mensaje de error al usuario. Esto preserva la integridad de los datos históricos del sistema.

Validación de campos obligatorios: Los campos task_name, when_created, due_date y ponderation son obligatorios tanto en validación del lado del cliente (atributo required en HTML) como en el controlador. La ponderación debe estar en el rango 0-100%.

Mensajes de feedback: El sistema implementa un mecanismo de notificación al usuario mediante query parameters en la URL. Tras crear, editar o borrar una tarea, la aplicación redirige a /tasks?message=... o /tasks?error=..., mostrando un mensaje de confirmación o error en la parte superior de la lista.

Reto Técnico: Corrección de Imports

Durante la integración del CRUD de tareas, se detectó un error sistemático en múltiples routers: varios archivos importaban el middleware requireRole en lugar de requireRoleView. Este error provocaba fallos en tiempo de ejecución (ReferenceError: requireRole is not defined).

Archivos afectados:
- viewRouterUser.js
- viewRouterTasks.js
- viewRouterEvaluations.js

Solución aplicada: Se corrigieron todos los imports mediante sustitución global:

```
sed -i 's/requireRole/requireRoleView/g' src/routes/views/*.js
```

Esta corrección garantizó la coherencia del sistema de autorización en toda la aplicación.

6. Conclusiones y Futuros Pasos

El desarrollo de Rubrika ha permitido consolidar conocimientos sobre arquitectura MVC, gestión de bases de datos relacionales con múltiples tablas intermedias y buenas prácticas de seguridad en aplicaciones web.

El sistema es funcional y cumple con lo siguiente: autenticación con roles, base de datos relacional con 8 tablas relacionadas, endpoints operativos con operaciones de escritura, vistas con Pug y API RESTful independiente.

Principales aprendizajes del proyecto:

- La importancia de fijar las versiones de las dependencias en package.json para evitar instalaciones de versiones con breaking changes.
- La diferencia entre bind mounts y volúmenes Docker nombrados y sus implicaciones en la gestión del ciclo de vida de los datos.
- La necesidad de verificar los nombres exactos de las variables de entorno en el código frente al archivo .env.
- El valor de trabajar con ramas por funcionalidad y Pull Requests para mantener la rama principal estable.

Mejoras para la v2.0

- Implementar autenticación con JWT para dar soporte a un cliente móvil o SPA en el futuro.
- Añadir paginación y filtrado en los endpoints de listado para mejorar el rendimiento.
- Documentar la API con Swagger/OpenAPI para facilitar la integración con otros sistemas.
- Implementar tests automatizados con Jest y Supertest.